Objetivo

Modelar y simular la caída vertical de un cuerpo en presencia de resistencia aerodinámica cuadrática.

El proyecto estudia:

- evolución de la altura
- evolución de la velocidad
- velocidad terminal
- impacto con el suelo
- comparación entre distintos cuerpos

Sistema físico

Se considera un cuerpo en caída vertical bajo la acción de dos fuerzas principales:

Peso

$$F_g = mg$$

Resistencia aerodinámica

$$F_d = \frac{1}{2}\rho C_d A v^2$$

donde:

- m: masa [kg]
- g: aceleración de la gravedad [m/s²]
- ρ: densidad del aire [kg/m³]
- C_d: coeficiente de arrastre [-]
- A: área frontal [m²]
- v: velocidad

Convención de signos

Se define:

- $h(t)$: altura sobre el suelo
- $v(t)$: velocidad positiva hacia abajo

Por tanto:

- si el cuerpo cae, $v > 0$
- la altura disminuye con el tiempo

Modelo matemático

La ecuación dinámica es:

$$m\frac{dv}{dt} = mg - \frac{1}{2}\rho C_d A v^2$$

Y el modelo en variables de estado queda:

$$\frac{dh}{dt} = -v$$

$$\frac{dv}{dt} = g - \frac{1}{2m}\rho C_d A v^2$$

Interpretación física del modelo

Régimen inicial

Cuando la velocidad es pequeña, el arrastre es casi despreciable:

$$\frac{1}{2}\rho C_d A v^2 \approx 0$$

Entonces:

$$\frac{dv}{dt} \approx g$$

Es decir, al principio la caída se parece a una caída libre ideal.

Régimen intermedio

Al aumentar la velocidad, el término de arrastre crece como v^2, por lo que la aceleración disminuye progresivamente.

Régimen terminal

La velocidad terminal aparece cuando el arrastre iguala al peso:

$$mg = \frac{1}{2}\rho C_d A v_t^2$$

Despejando:

$$v_t = \sqrt{\frac{2mg}{\rho C_d A}}$$

En ese punto:

$$\frac{dv}{dt} = 0$$

y la velocidad deja de aumentar.

Caso base

Parámetros del caso base:

- $m = 80.0 \text{ kg}$
- $g = 9.81 \text{ m/s}^2$
- $\rho = 1.225 \text{ kg/m}^3$
- $C_d = 1.0$
- $A = 0.7 \text{ m}^2$
- $h_0 = 20.0 \text{ m}$
- $v_0 = 0.0 \text{ m/s}$

Resultados del caso base

En el caso base:

- la altura disminuye desde 20 m hasta 0 m
- la velocidad aumenta desde 0 m/s
- el impacto ocurre antes de alcanzar la velocidad terminal

Por tanto:

$$v_{\text{impacto}} < v_t$$

Esto demuestra que la existencia de una velocidad terminal no implica que siempre llegue a alcanzarse; depende de la altura disponible y del tiempo de caída.

Evento de impacto con el suelo

Para evitar alturas negativas, se ha implementado una condición de parada cuando:

$$h = 0$$

La simulación se detiene exactamente en el impacto, permitiendo obtener:

- tiempo de impacto
- velocidad de impacto
- altura final físicamente coherente

Estudio de velocidad terminal

Se ha comparado la dinámica de distintos cuerpos:

- Human
- Squirrel compact
- Squirrel open posture
- Dense compact object

Idea clave

La velocidad terminal depende de la relación entre:

- masa
- área frontal
- coeficiente de arrastre

según:

$$v_t = \sqrt{\frac{2mg}{\rho C_d A}}$$

Interpretación comparativa

Human

- velocidad terminal media-alta
- tarda más en acercarse al régimen casi constante

Squirrel compact

- menor velocidad terminal
- alcanza antes un régimen casi constante

Squirrel open posture

- aún menor velocidad terminal

- la postura aumenta el frenado aerodinámico

Dense compact object

- velocidad terminal muy alta
- el aire lo frena relativamente poco

Conclusión física

La velocidad en caída con aire no crece indefinidamente.

El arrastre aerodinámico aumenta con v^2 y termina equilibrando el peso, lo que limita la velocidad máxima alcanzable.

La simulación confirma que:

- caídas cortas pueden terminar antes de llegar a v_t
- caídas largas convergen hacia la velocidad terminal
- aumentar C_d o A reduce la velocidad terminal
- aumentar la relación m/A tiende a elevarla

Valor del proyecto dentro del portfolio

Este proyecto aporta:

- modelado físico básico
- formulación matemática de un sistema no lineal
- integración numérica de EDOs
- detección de eventos
- visualización de resultados
- interpretación técnica de parámetros físicos

Es una base sólida para proyectos posteriores de:

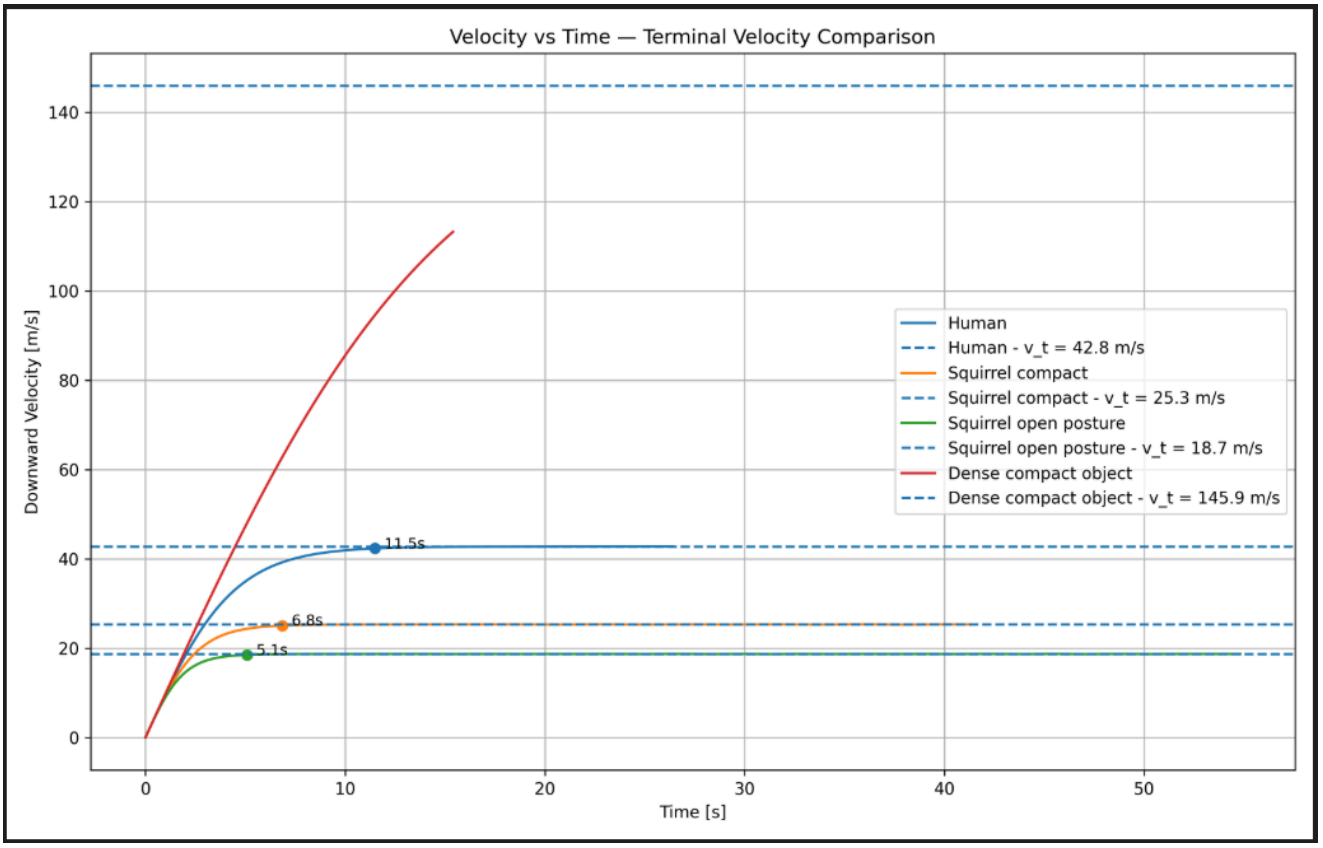
- balística
- descenso atmosférico
- paracaidismo simplificado
- dinámica vertical en fluidos
- simulación aeroespacial introductoria

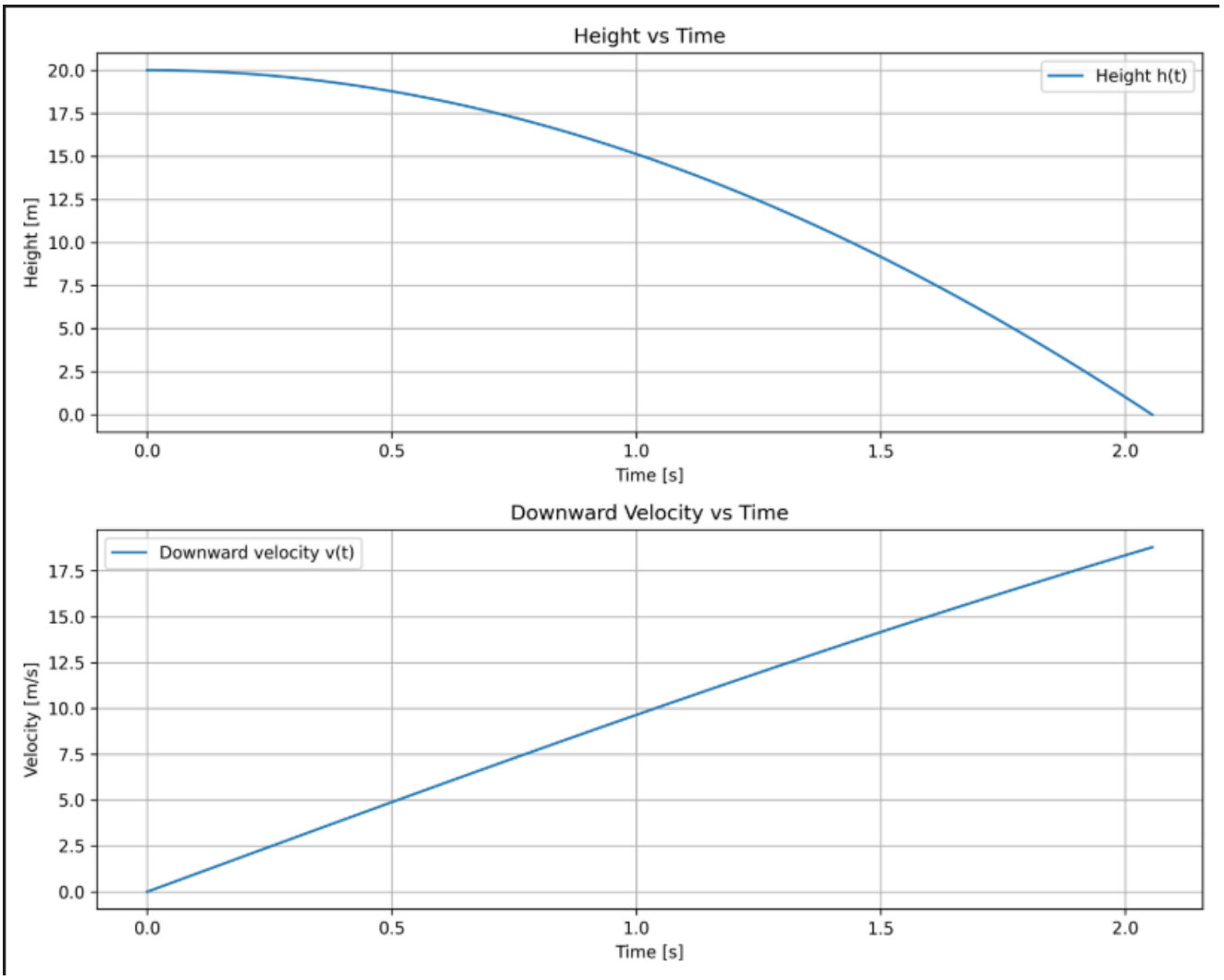
Limitaciones del modelo

- movimiento solo en 1D vertical
- densidad del aire constante
- C_d constante
- área frontal constante
- sin viento
- sin rotación
- sin cambios de masa
- sin efectos de compresibilidad

Mejoras futuras

- densidad del aire variable con la altura
- C_d dependiente de la velocidad
- área frontal variable con la postura
- exportación de resultados a CSV
- barridos paramétricos
- animación de la caída
- comparación con soluciones analíticas
- ejecución más empaquetada desde raíz del proyecto





Resumen del código Python — Free Fall with Drag

Objetivo del código

Implementar una simulación numérica de la caída vertical de un cuerpo con resistencia aerodinámica cuadrática, incluyendo:

- definición del modelo físico
- integración numérica
- detección del impacto con el suelo
- representación gráfica
- análisis comparativo de velocidad terminal

Estructura de archivos Python

```
src/
├── __init__.py
├── model.py
├── simulation.py
```

```
├── plots.py
└── analysis_terminal_velocity.py
```

`model.py`

Función principal

```
free_fall_with_drag(...)
```

Responsabilidad

Define el sistema de ecuaciones diferenciales que modela la caída.

Variables de estado

- `h` : altura sobre el suelo
- `v` : velocidad positiva hacia abajo

Ecuaciones implementadas

$$\frac{dh}{dt} = -v$$

$$\frac{dv}{dt} = g - \frac{1}{2m}\rho C_d A v^2$$

Lógica del código

1. recibe el tiempo `t` y el estado `state = [h, v]`
2. separa la altura y la velocidad
3. calcula la fuerza de arrastre:

$$F_d = \frac{1}{2}\rho C_d A v^2$$

4. calcula las derivadas:
   - `dh_dt = -v`
   - `dv_dt = g - drag_force / m`
5. devuelve `[dh_dt, dv_dt]`

Idea clave

Este archivo contiene **solo el modelo matemático**, sin ejecutar la simulación.

`simulation.py`

Función principal

```
run_simulation(...)
```

Responsabilidad

Ejecuta la integración numérica del sistema usando `solve_ivp`.

Elementos principales

1. Evento de impacto

Se define una función:

```
hit_ground_event(...)
```

que devuelve la altura `h`.

Cuando:

$$h = 0$$

el solver detecta el impacto con el suelo y detiene la simulación.

2. Configuración del evento

```
hit_ground_event.terminal = True
hit_ground_event.direction = -1
```

- `terminal = True` → detiene la integración
- `direction = -1` → detecta el cruce cuando la altura pasa de positiva a negativa

3. Integración numérica

Se usa:

```
solve_ivp(...)
```

con:

- función del modelo
- intervalo temporal
- condiciones iniciales
- parámetros físicos

- función de evento

4. Corrección del punto final

Como el evento puede detectarse entre dos puntos de `t_eval`, se añade explícitamente el punto exacto de impacto al resultado final.

Salida de `run_simulation`

Devuelve un diccionario con:

- `t` → vector de tiempo corregido
- `y` → estados corregidos
- `raw_solution` → solución original de SciPy
- `impact_detected` → indica si hubo impacto
- `impact_time` → tiempo exacto de impacto
- `impact_state` → estado exacto en el impacto

Idea clave

Este archivo contiene la **lógica de simulación**, no las gráficas.

`plots.py`

Función principal

```
plot_results()
```

Responsabilidad

Generar y guardar las gráficas del caso base.

Flujo del código

1. llama a `run_simulation(...)`
2. extrae:
   - `t`
   - `h`
   - `v`
3. crea la carpeta `results/` si no existe
4. genera una figura con dos subgráficas:
   - altura vs tiempo
   - velocidad vs tiempo

5. guarda la figura en:

```
results/free_fall_base_case.png
```

6. muestra la figura por pantalla

Idea clave

Este archivo separa claramente la parte de **visualización** del resto del proyecto.

`analysis_terminal_velocity.py`

Objetivo

Comparar varios cuerpos y estudiar la velocidad terminal.

Funciones principales

`terminal_velocity(...)`

Calcula la velocidad terminal teórica:

$$v_t = \sqrt{\frac{2mg}{\rho C_d A}}$$

`time_to_reach_fraction(...)`

Calcula el primer instante en que la velocidad alcanza una fracción dada de la velocidad terminal, por ejemplo:

$$v(t) \geq 0.99, v_t$$

`analyze_case(...)`

Para un cuerpo dado:

1. ejecuta la simulación
2. extrae altura y velocidad
3. calcula la velocidad terminal teórica
4. calcula el tiempo para llegar al 99% de v_t
5. imprime resultados
6. devuelve un diccionario con los datos del caso

`plot_comparison(...)`

Genera una gráfica comparativa de velocidad vs tiempo para varios cuerpos.

Además:

- dibuja una línea horizontal para cada velocidad terminal
- marca el instante en que se alcanza el 99% de v_t
- guarda la figura en:

```
results/terminal_velocity_comparison.png
```

`main()`

Define y analiza varios casos:

- Human
- Squirrel compact
- Squirrel open posture
- Dense compact object

Idea clave

Este archivo añade una capa de **análisis físico comparativo**, no solo simulación básica.

Flujo general del proyecto en Python

Paso 1 — modelo físico

`model.py` define las ecuaciones diferenciales.

Paso 2 — simulación

`simulation.py` resuelve numéricamente el sistema y detecta el impacto.

Paso 3 — visualización básica

`plots.py` representa el caso base.

Paso 4 — análisis comparativo

`analysis_terminal_velocity.py` compara varios cuerpos y estudia la velocidad terminal.

Bibliotecas utilizadas

NumPy

Se usa para:

- crear vectores temporales
- trabajar con arrays
- operaciones numéricas
- cálculo de raíz cuadrada
- búsqueda de índices

SciPy

Se usa para:

- resolver el sistema dinámico mediante `solve_ivp`
- detectar eventos de impacto

Matplotlib

Se usa para:

- crear las gráficas
- añadir títulos, etiquetas y leyendas
- guardar resultados en archivos `.png`

pathlib

Se usa para:

- construir rutas de forma robusta
- localizar la carpeta `results/`

Buenas prácticas aplicadas

- separación por responsabilidades
- funciones pequeñas y reutilizables
- nombres claros de variables
- documentación con docstrings
- generación automática de resultados gráficos
- detección de eventos físicos
- estructura escalable para portfolio

Conclusión sobre el código

El proyecto está organizado de forma modular y profesional:

- el modelo físico está separado de la simulación
- la simulación está separada de la visualización
- el análisis comparativo está separado del caso base

Esto facilita:

- entender el código
- mantenerlo
- ampliarlo en proyectos futuros
- reutilizar partes del flujo en otros sistemas dinámicos